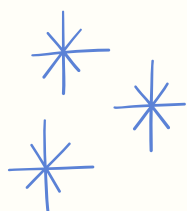




Tecnológico Nacional de México



Instituto Tecnológico Superior de Mulegé



ingeniería //

DEL CONOCIMIENTO

Ing. Rafael Chaidez Rosas



Ingeniería en Tecnologías de la Información y
comunicaciones

Arianna Aguilar Martínez
[221061094]



Contenido

Proyecto Final - Programación para IoT	3
1. Planteamiento del problema.....	3
2. Objetivos	3
3. Descripción de la solución propuesta.....	4
4. Diseño del sistema	5
4.1 Diseño de interfaz (Sketch).....	5
4.2 Casos de uso	5
4.3 Diagramas de actividad	8
4.4 Diagramas de secuencia	10
4.5 Diagrama de clases	11
4.6 Mapa del sitio.....	12
4.7 Diseño de la API REST	13
Registrar lectura	13
Obtener lecturas.....	13
4.8 Diseño de Base de Datos	15
Tabla: lecturas	15
Tabla: alertas.....	15
Tabla: usuarios	16
4.9 Diseño electrónico	16
Conexiones.....	16
Diagrama	18
5. Tecnologías utilizadas	19
Hardware	19
Software.....	19
Backend	19
Frontend.....	19
Base de Datos.....	19
Desarrollo.....	20
Frameworks y Librerías	20
CodeIgniter 4.....	20

Bootstrap 5	20
Apache ECharts	20
Protocolos de Comunicación	21
WiFi	21
HTTP/HTTPS	21
REST API	21
JSON.....	21
Arquitectura de Software	21
Capa de Presentación.....	21
6. Variables	22

Proyecto Final - Programación para IoT

1. Planteamiento del problema

Actualmente muchas personas presentan problemas de sueño sin ser conscientes de ello, lo que afecta su salud física, emocional y rendimiento académico o laboral. Los dispositivos comerciales para monitoreo del sueño suelen ser costosos o requieren el uso de accesorios corporales incómodos.

Además, factores ambientales como la temperatura, humedad, ruido y movimiento durante la noche influyen directamente en la calidad del descanso, pero normalmente no son monitoreados de forma continua.

Por ello surge la necesidad de desarrollar un sistema inteligente basado en IoT que permita monitorear las condiciones ambientales del dormitorio y generar recomendaciones para mejorar la calidad del sueño de manera económica, accesible y no invasiva.

2. Objetivos

Objetivo General

Desarrollar un sistema inteligente basado en IoT capaz de monitorear las condiciones ambientales y físicas relacionadas con el descanso nocturno para evaluar la calidad del sueño y generar recomendaciones automáticas.

Objetivos Específicos

- Medir temperatura y humedad ambiental mediante sensores.
- Detectar niveles de ruido durante la noche.
- Registrar movimientos corporales nocturnos.
- Calcular un Índice de Calidad del Sueño (ICS).
- Almacenar los datos en una base de datos.
- Mostrar información mediante un dashboard web.

- Generar alertas y recomendaciones automáticas.
- Implementar indicadores visuales mediante LED RGB y pantalla OLED.

3. Descripción de la solución propuesta

La solución consiste en un sistema IoT basado en un ESP32 encargado de recopilar información de diversos sensores ubicados en una habitación.

Los datos obtenidos serán enviados mediante WiFi a una API REST, donde serán almacenados en una base de datos MySQL.

Posteriormente, un dashboard web permitirá visualizar:

1. Temperatura.
2. Humedad.
3. Nivel de ruido.
4. Movimiento detectado.
5. Índice de Calidad del Sueño.
6. Historial de registros.
7. Alertas y recomendaciones.

Además, el sistema utilizará actuadores para informar el estado del descanso:

- LED RGB:
 - Verde → Sueño excelente.
 - Amarillo → Sueño regular.
 - Rojo → Sueño deficiente.
- Pantalla OLED:
 - Recomendaciones.
 - Alertas.
 - Estado actual.

4. Diseño del sistema

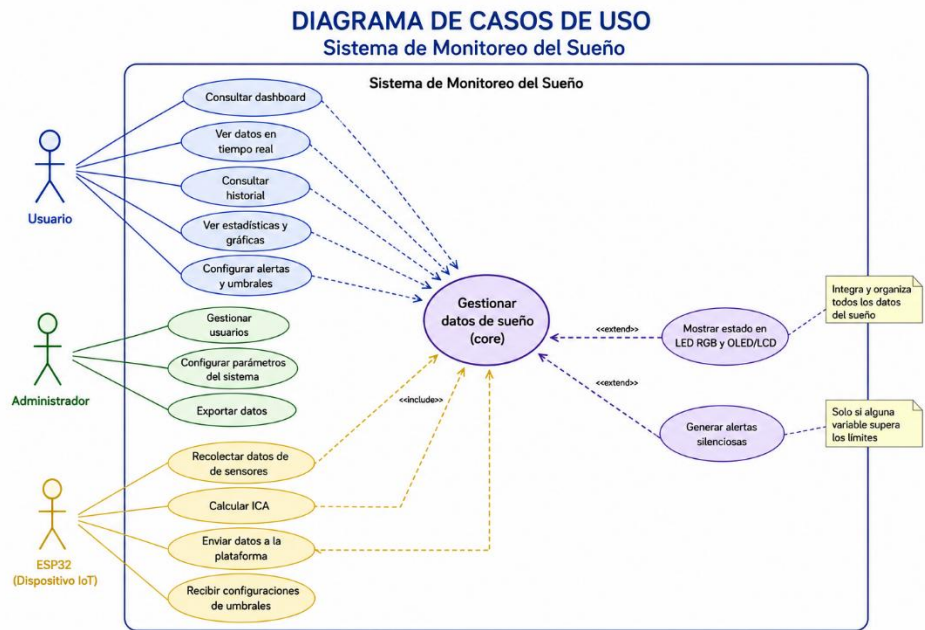
4.1 Diseño de interfaz (Sketch)



4.2 Casos de uso

Incluir:

- Diagrama de casos de uso.



- Descripción de cada caso de uso.

ID	Caso de uso	Actor principal	Descripción breve
CU-01	Consultar dashboard	Usuario	Visualiza el panel principal con resumen del sueño e ICA actual
CU-02	Ver datos en tiempo real	Usuario	Observa lecturas de sensores actualizándose automáticamente
CU-03	Consultar historial	Usuario	Busca registros pasados por fecha o rango
CU-04	Ver estadísticas y gráficas	Usuario	Muestra gráficas de tendencias y promedios semanales
CU-05	Configurar alertas y umbrales	Usuario	Define límites de temperatura, ruido y humedad para alertas
CU-06	Gestionar usuarios	Administrador	Crea, edita o elimina usuarios del sistema
CU-07	Configurar parámetros del sistema	Administrador	Ajusta frecuencia de envío, umbrales globales y opciones del sistema
CU-08	Exportar datos	Administrador	Descarga registros en CSV/JSON para respaldo o análisis
CU-09	Recolectar datos de sensores	ESP32	Lee sensores DHT11, KY-038 y SW-420
CU-10	Calcular ICA	ESP32	Calcula Índice de Calidad del Sueño (0-100) con 4 variables
CU-11	Enviar datos a la plataforma	ESP32	Envía JSON por HTTP POST al servidor
CU-12	Recibir configuraciones de umbrales	ESP32	Recibe nuevos límites de alerta desde el servidor
CU-13	Gestionar datos de sueño	Sistema (Core)	Integra y organiza toda la información del sueño
CU-14	Mostrar estado en LED RGB y OLED/LCD	ESP32	Cambia color LED y muestra mensajes en pantalla según ICA
CU-15	Generar alertas silenciosas	Sistema	Notifica sin sonido cuando una variable supera los límites

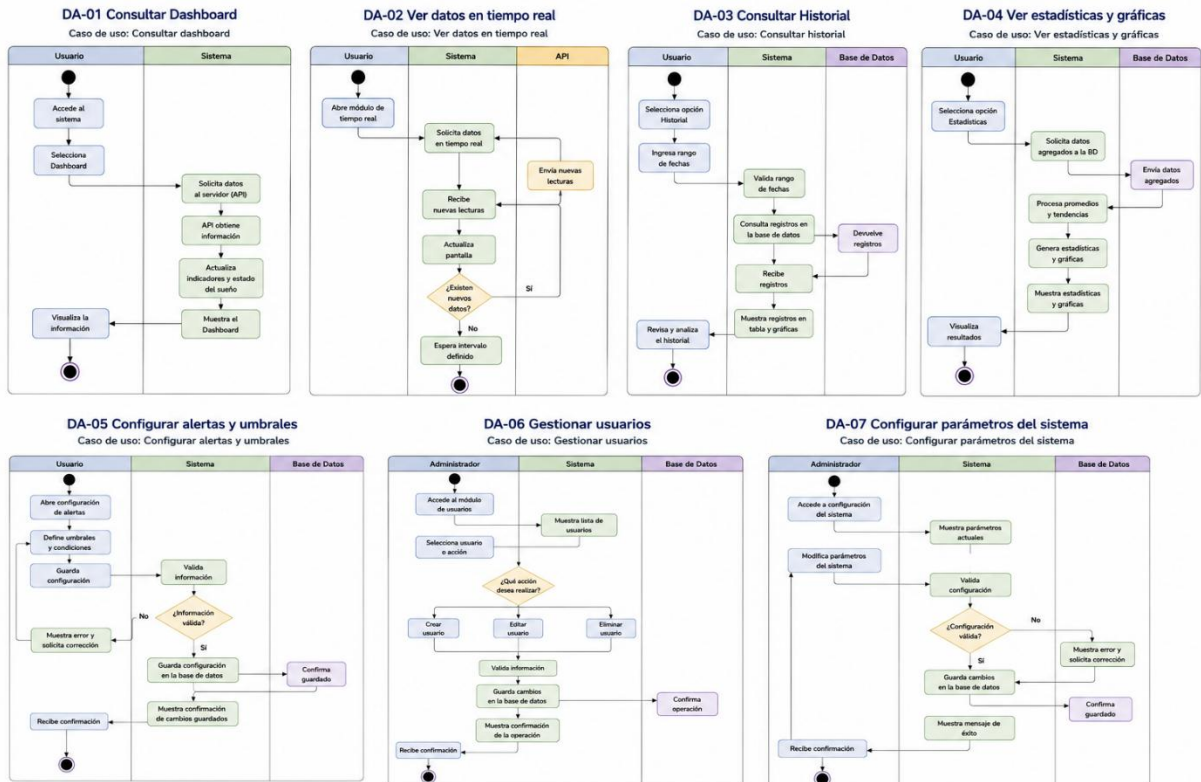
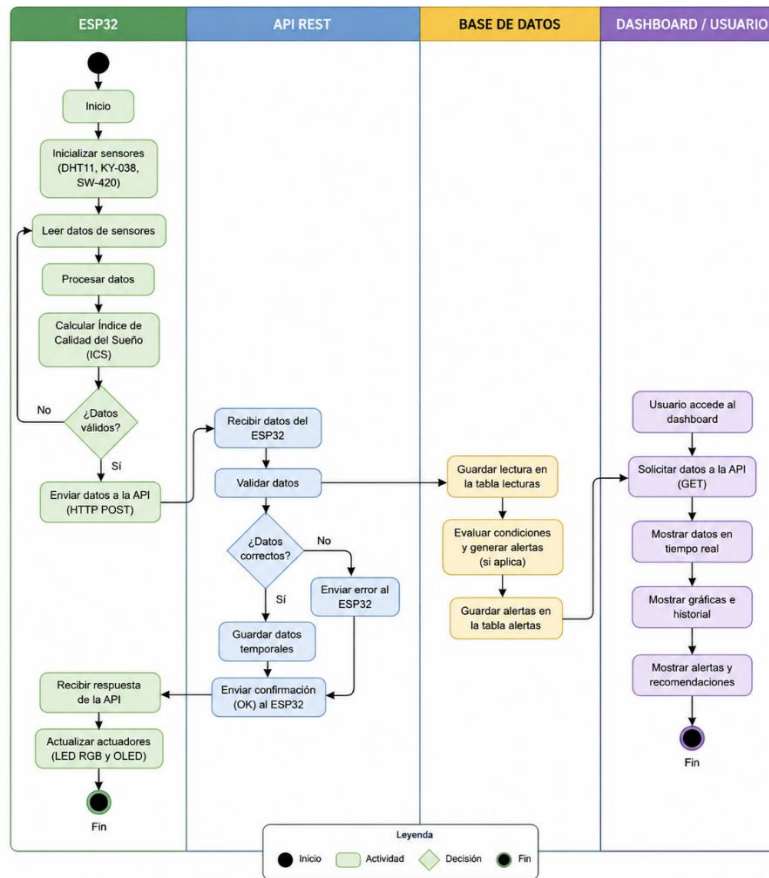
- Actores involucrados.

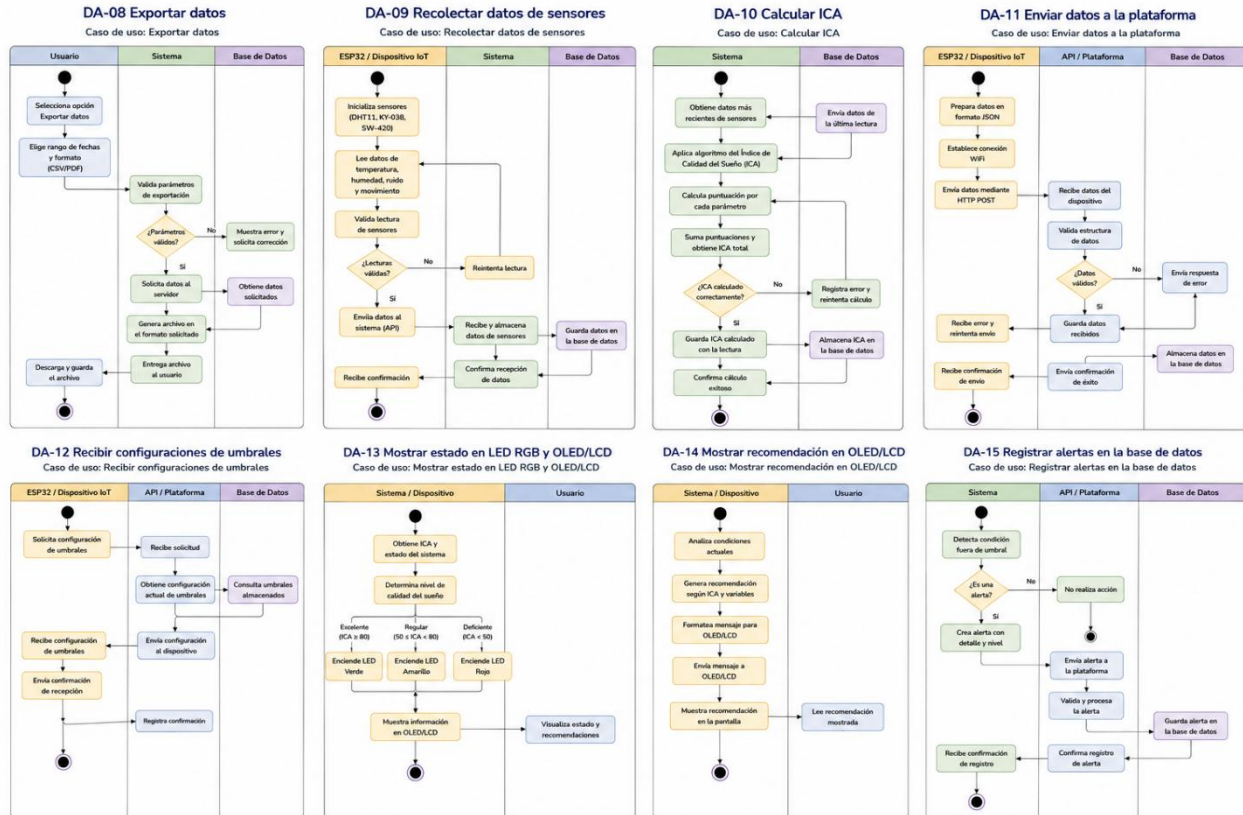
Actor	Rol	Responsabilidades principales	Casos de uso asociados
Usuario	Persona que consulta el sistema	Visualizar información del sueño, configurar sus alertas	CU-01, CU-02, CU-03, CU-04, CU-05
Administrador	Gestor del sistema	Administrar usuarios, parámetros globales y datos	CU-06, CU-07, CU-08
ESP32 (IoT Device)	Dispositivo físico	Capturar sensores, calcular ICA, enviar datos, actuar	CU-09, CU-10, CU-11, CU-12, CU-14
Sistema (Core)	Lógica central automática	Integrar datos, generar alertas (extend)	CU-13, CU-15

- Relaciones Entre Casos De Uso (Include / Extend)

Caso base	Relación	Caso relacionado	Condición
CU-13 (Gestionar datos de sueño)	<<include>>	CU-11 (Enviar datos a la plataforma)	Siempre ocurre
CU-13 (Gestionar datos de sueño)	<<extend>>	CU-15 (Generar alertas silenciosas)	Solo si hay anomalía
CU-13 (Gestionar datos de sueño)	<<extend>>	CU-14 (Mostrar estado LED/OLED)	Siempre que hay actualización de ICA

4.3 Diagramas de actividad





Usuario

1. Consultar dashboard
2. Ver datos en tiempo real
3. Consultar historial
4. Ver estadísticas y gráficas
5. Configurar alertas y umbrales

Administrador

6. Gestionar usuarios
7. Configurar parámetros del sistema
8. Exportar datos

ESP32

9. Recolectar datos de sensores
10. Calcular ICA
11. Enviar datos a la plataforma
12. Recibir configuraciones de umbrales

Casos Core / Extend

13. Gestionar datos de sueño (core)
14. Mostrar estado en LED RGB y OLED/LCD
15. Generar alertas silenciosas

4.4 Diagramas de secuencia

Representar la comunicación entre los componentes del sistema.

Usuario → Dashboard → API → Base de Datos

Usuario solicita información.

Dashboard consulta API.

API consulta Base de Datos.

Base de Datos responde.

Dashboard muestra información.

ESP32 → API → Base de Datos

ESP32 recopila datos.

ESP32 envía datos.

API recibe datos.

Base de Datos almacena registros.

API responde confirmación.

Usuario → API → ESP32

Usuario configura umbrales.

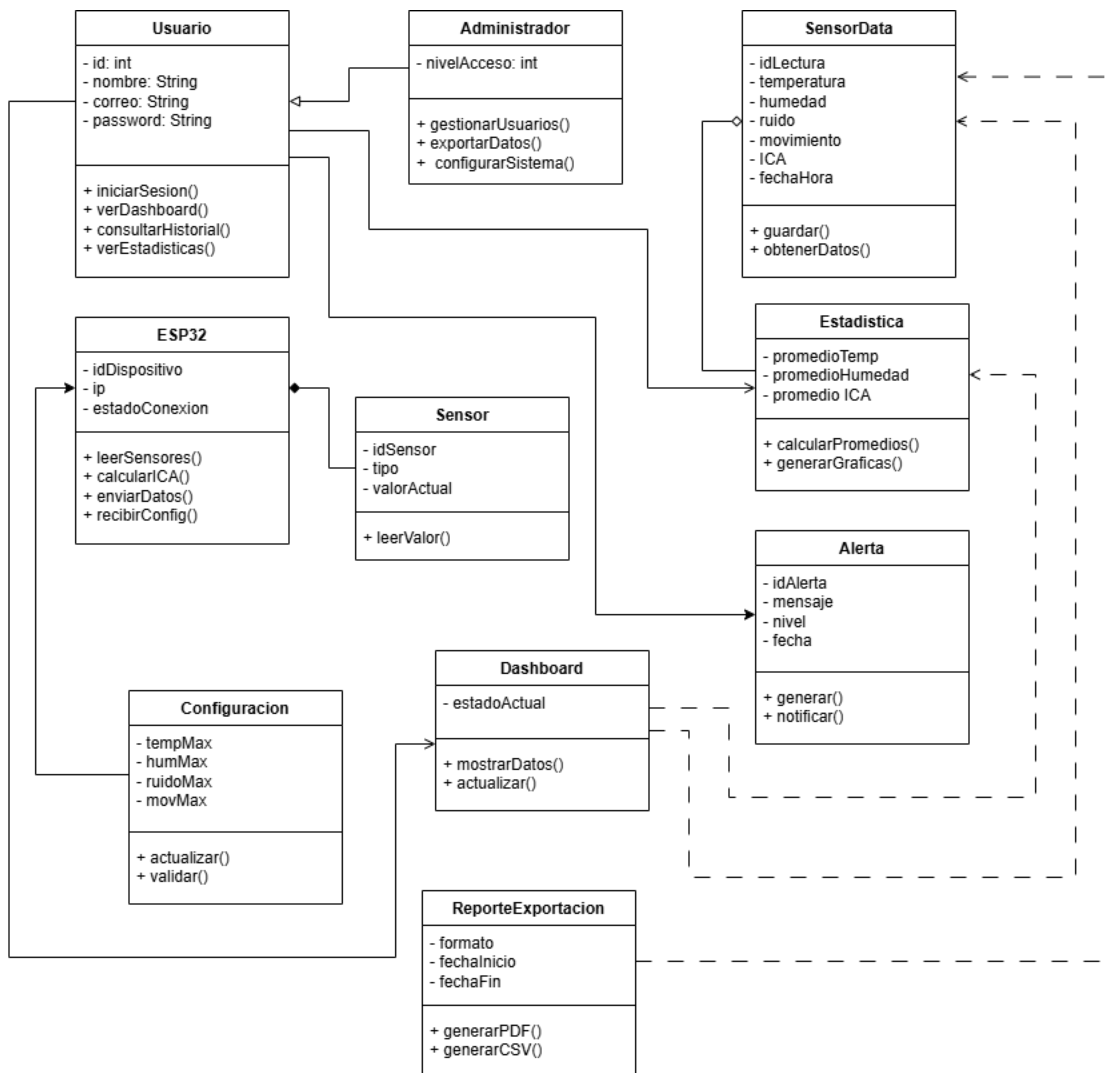
API almacena configuración.

ESP32 solicita configuración.

API envía configuración.

ESP32 actualiza parámetros.

4.5 Diagrama de clases

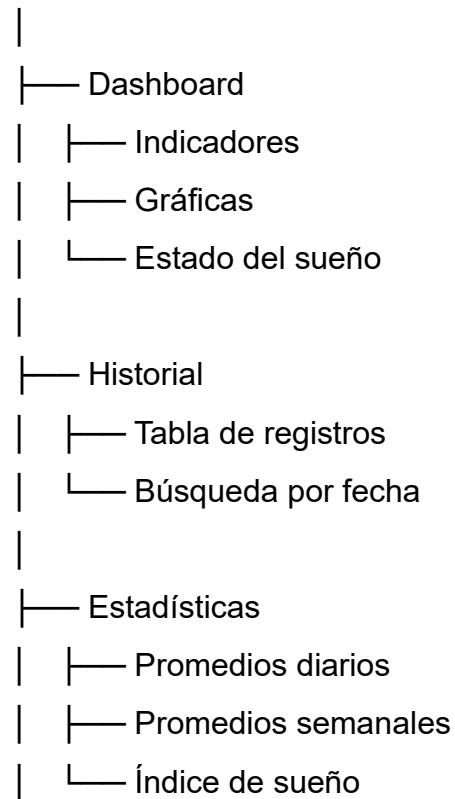


Clase	Tipo
<i>Usuario</i>	Entidad
<i>Administrador</i>	Entidad
<i>ESP32</i>	Dispositivo IoT
<i>Sensor</i>	Abstracta
<i>DHT11</i>	Sensor
<i>KY038</i>	Sensor
<i>SW420</i>	Sensor
<i>SensorData</i>	Entidad
<i>Dashboard</i>	Boundary
<i>Estadistica</i>	Control
<i>Alerta</i>	Entidad
<i>Configuracion</i>	Control
<i>ReporteExportacion</i>	Control

4.6 Mapa del sitio

Representar la estructura de navegación de la aplicación web.

Inicio



|
|— Alertas
|
|— Acerca de

4.7 Diseño de la API REST

Registrar lectura

POST /api/lecturas

Petición

```
{  
  "temperatura":24,  
  "humedad":55,  
  "ruido":35,  
  "movimiento":0,  
  "indice_sueno":87  
}
```

Respuesta

```
{  
  "status":"ok",  
  "mensaje":"Lectura registrada"  
}
```

Obtener lecturas

GET /api/lecturas

Respuesta

```
[  
  {  
    "temperatura":24,
```

```
"humedad":55,  
"ruido":35  
}  
]
```

Obtener estadísticas

GET /api/promedios

Respuesta

```
{  
  "temp_promedio":24.5,  
  "humedad_promedio":58,  
  "indice_promedio":82  
}
```

Obtener alertas

GET /api/alertas

Respuesta

```
[  
  {  
    "mensaje":"Ruido elevado",  
    "nivel":"medio"  
  }  
]
```

4.8 Diseño de Base de Datos

Tabla: lecturas

Campo	Tipo
id	INT AUTO_INCREMENT
temperatura	FLOAT
humedad	FLOAT
ruido	FLOAT
movimiento	INT
indice_sueno	FLOAT
fecha	DATETIME

Tabla: alertas

Campo	Tipo
id	INT AUTO_INCREMENT
mensaje	VARCHAR(255)
nivel	VARCHAR(50)
fecha	DATETIME

Tabla: usuarios

Campo	Tipo
id	INT AUTO_INCREMENT
nombre	VARCHAR(100)
correo	VARCHAR(100)
password	VARCHAR(255)

4.9 Diseño electrónico

Componentes

- ESP32
- DHT11
- KY-037
- SW-420
- LED RGB
- Pantalla OLED I2C

Conexiones

OLED SSD1306

Pin del módulo	ESP32 DevKit V1
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

DHT11 (Temperatura y Humedad)

Pin del módulo	ESP32 DevKit V1
VCC	3.3V
GND	GND
DATA	GPIO 4

KY-037 (Sensor de sonido)

Pin del módulo	ESP32 DevKit V1
VCC	3.3V
GND	GND
DO	GPIO 34

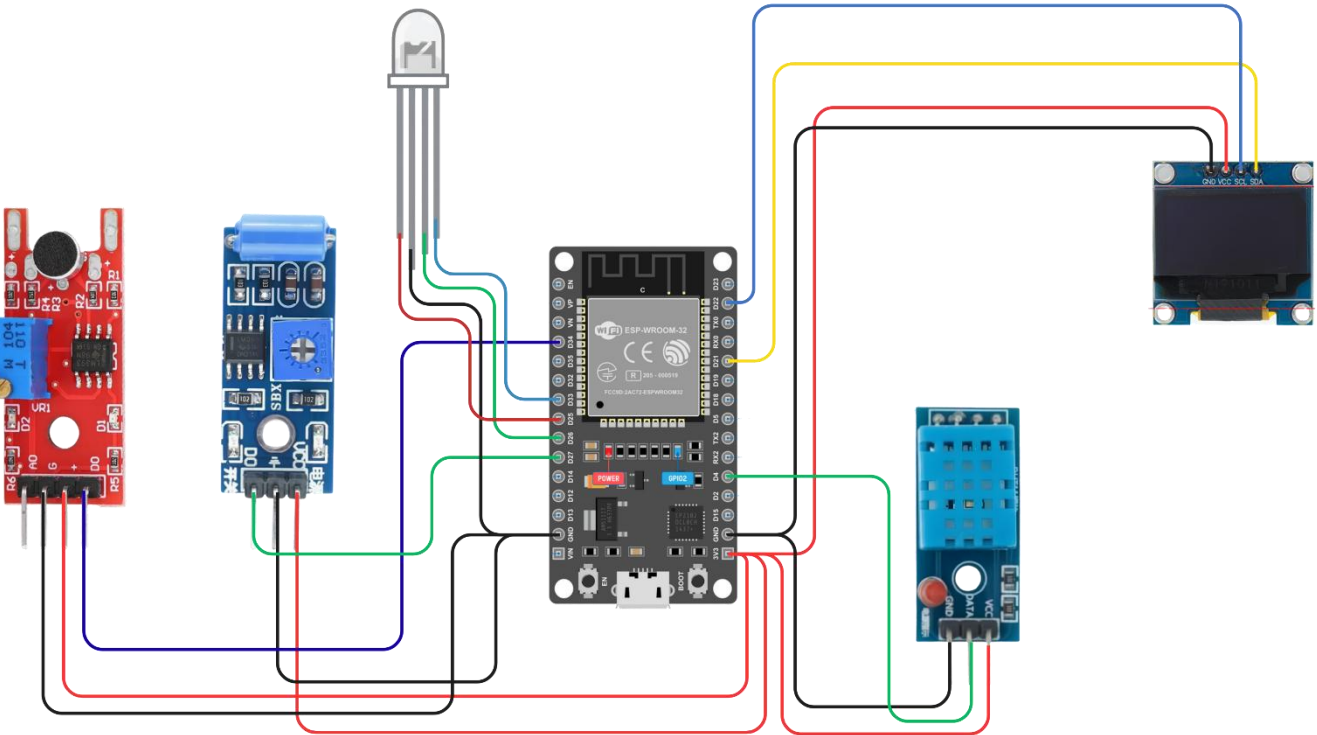
SW-420 (Sensor de vibración)

Pin del módulo	ESP32 DevKit V1
VCC	3.3V
GND	GND
DO	GPIO 27

LED RGB

Color / Pin	Pin del módulo LED RGB	ESP32 DevKit V1
Rojo	R	GPIO 25
Verde	G	GPIO 26
Azul	B	GPIO 33
Común	GND	GND

Diagrama



5. Tecnologías utilizadas

Hardware

- ESP32
- Sensor DHT11 (temperatura y humedad)
- Sensor KY-038 (sonido)
- Sensor SW-420 (vibración/movimiento)
- Pantalla OLED I2C
- LED RGB

Software

Backend

- **PHP 8**
- **CodeIgniter 4**
 - Framework MVC utilizado para el desarrollo de la API REST y la lógica del sistema.
 - Gestión de rutas, controladores, modelos y consultas a la base de datos.

Frontend

- HTML5
- CSS3
- JavaScript
- Bootstrap 5
- Apache ECharts

Base de Datos

- MySQL
- phpMyAdmin

Desarrollo

- Visual Studio Code
- Arduino IDE
- XAMPP

Frameworks y Librerías

CodeIgniter 4

Framework PHP basado en el patrón MVC utilizado para:

- Desarrollo de la API REST.
- Gestión de usuarios.
- Manejo de rutas.
- Comunicación con la base de datos.
- Generación de estadísticas.
- Exportación de datos.

Bootstrap 5

Utilizado para:

- Diseño responsivo.
- Componentes visuales.
- Formularios.
- Dashboard administrativo.

Apache ECharts

Utilizado para:

- Gráficas de temperatura.
- Gráficas de humedad.
- Gráficas de ruido.
- Gráficas del Índice de Calidad del Sueño (ICS).
- Estadísticas históricas.

Protocolos de Comunicación

WiFi

Permite la conexión del ESP32 con el servidor.

HTTP/HTTPS

Utilizado para la comunicación entre el ESP32 y la API REST.

REST API

Arquitectura utilizada para el intercambio de información entre:

- ESP32
- Servidor
- Dashboard Web

JSON

Formato utilizado para enviar y recibir datos entre los componentes del sistema.

Arquitectura de Software

El sistema utiliza una arquitectura de tres capas:

Capa de Presentación

- HTML5
- CSS3
- Bootstrap
- JavaScript
- Apache ECharts

Capa de Negocio

- PHP
- CodeIgniter 4
- Controladores
- Servicios
- API REST

Capa de Datos

- MySQL
- Modelos de CodeIgniter 4

6. Variables

El Índice de Calidad del Sueño (ICA) que se planea calcular es una **variable compuesta**. No es una variable en sí misma, sino el **resultado de una fórmula** que combina las variables mencionadas.

Dimensión Estándar	Variable Equivalente en el Proyecto	Descripción
Eficiencia del sueño	movimiento + tiempo en cama	Porcentaje de tiempo que pasas dormido realmente mientras estás en la cama . Si tu sensor detecta mucho movimiento, la eficiencia baja.
Duración del sueño	Tiempo (inferido) + movimiento	Tiempo total que pasas dormido (restando los períodos de movimiento nocturno) .
Fragmentación del sueño	movimiento (frecuencia)	Mide cuántas veces te despiertas o tienes movimientos bruscos durante la noche .
Latencia del sueño	movimiento + ruido	Tiempo que tardas en quedarte dormido después de acostarte (detección de cese de ruido/movimiento) .
Entorno (Factores externos)	Temperatura, Humedad, Ruido	Evalúan las condiciones ideales para el descanso .

Propuesta de Fórmula del ICA (basada en el PSQI y Actigrafía):

Puedes calcular tu ICA como un porcentaje de 0 a 100 basándote en rangos "saludables" estándar:

1. **Duración (30% del ICA):** Se considera saludable dormir entre 6 y 8 horas .
 - *Lectura ideal: $6h < TST \text{ (Total Sleep Time)} < 8h$*
2. **Eficiencia (30% del ICA):** Se considera buena si supera el 85% .
 - *Fórmula: $(\text{Tiempo_dormido} / \text{Tiempo_en_cama}) \times 100$ *
3. **Estabilidad (20% del ICA):** (Inferido por movimiento) Bajo movimiento o movimientos estables indican un sueño reparador .
4. **Entorno (20% del ICA):** Rangos ideales de temperatura (18-22°C), humedad (40-60%) y bajo ruido (<30dB).

Ventaja de este enfoque: El ICA simula los componentes principales del cuestionario PSQI adaptándolos a datos objetivos medibles por el ESP32, lo cual es una solución científicamente fundamentada.